

Sistemas Difusos: Diseño

Instrucciones. Contesta lo que se te pide. Utiliza los formatos de tabla que se te proporcionan para indicar las respuestas que así lo requieran.

Diseño

Diseña y **programa un sistema** difuso relacionado con videojuegos. Incluye en cada apartado los diferentes elementos de tu sistema.

Descripción del videojuego

El videojuego "Mario Kart" ha sido una popular serie de videojuegos de carreras que ha existido desde la niñez de varios usuarios. Se trata de un juego que combina la mecánica de carreras con objetos especiales (power-ups) para dar un poco más de versatilidad con una variedad de pistas, personajes y habilidades especiales para competir.

Los jugadores pueden elegir entre una amplia variedad de personajes y cada personaje tiene sus propias características y habilidades especiales que afectan el rendimiento en la pista. Igualmente, durante las carreras, los jugadores pueden recoger power-ups y objetos especiales que les proporcionan ventajas o les permiten obstaculizar a los oponentes siendo que cada uno hace funciones diferentes. Las pistas también tienen diseños diferentes y cada una presenta obstáculos únicos orientados al tema de estas.

En los modos clásicos del juego son un total de 3 vueltas para competir. El objetivo es acumular la mayor cantidad de puntos posibles (obtenidos en función a la posición de cada vuelta). Por ejemplo, alguien que llegó en 4° lugar en las 3 vueltas no tendrá los mismos puntos que alguien que llegó en 1° lugar en todas.



Descripción del sistema

El sistema propuesto predice la clasificación en la que podrías quedar dependiendo de las variables de entrada (1) velocidad del kart que manejas y (2) la distancia entre cada vuelta de la carrera (teniendo en cuenta que cada partida dura entre 15-30 minutos).

Variables

Indica qué variables tendrá tu sistema. Considera **dos** variables de entrada y **una** de salida. Especifica el tipo (entrada o salida) y rango que tendrán las variables.

Variable	Tipo (entrada/salida)	Rango
Velocidad del Kart	Entrada	0-150 km/hr
Distancia entre c/vuelta	Entrada	10-30 metros
Clasificaciones	Salida	1-12 puestos

Conjuntos difusos y funciones de membresía

Particiona cada variable en conjuntos difusos. Define cada conjunto utilizando funciones de membresía (tipo y parámetros).

Variable lingüística: **Velocidad del Kart**

Conjunto difuso	Función	Parámetros
Baja	Trapezoidal-R	c=25, d=60
Media	Trapezoidal	a=35, b=60, c=75, d=100
Alta	Trapezoidal-L	a=75, b=110

Variable lingüística: **Distancia entre c/vuelta**

Conjunto difuso	Función	Parámetros
Corto	Trapezoidal-R	c=5, d=10
Medio	Trapezoidal	a=7, b=10, c=12, d=20
Largo	Trapezoidal-L	a=12, b=25

Variable lingüística: **Clasificaciones**

Conjunto difuso	Función	Parámetros
Buena	Trapezoidal-R	c=2, d=4
Promedio	Trapezoidal	a=3, b=5, c=6, d=8
Mala	Trapezoidal-L	a=6, b=8

Base de reglas

1. Representa las reglas del sistema como una matriz.
2. Representa también las reglas en formato IF-THEN.

IF Velocidad es Baja y distancia es Corto THEN clasificación es Mala.

IF Velocidad es Baja y distancia es Medio THEN clasificación es Mala.

IF Velocidad es Baja y distancia es Largo THEN clasificación es Mala.

IF Velocidad es Media y distancia es Corto THEN clasificación es Buena.

IF Velocidad es Media y distancia es Medio THEN clasificación es Promedio.

IF Velocidad es Media y distancia es Largo THEN clasificación es Mala.

IF Velocidad es Alta y distancia es Corto THEN clasificación es Buena.

IF Velocidad es Alta y distancia es Medio THEN clasificación es Buena.

IF Velocidad es Alta y distancia es Largo THEN clasificación es Promedio.

Matriz

velocidad	distancia	Clasificación
Baja	Corto	Mala
Baja	Medio	Mala
Baja	Largo	Mala
Media	Corto	Buena
Media	Medio	Promedio
Media	Largo	Mala
Alta	Corto	Buena
Alta	Medio	Buena
Alta	Largo	Promedio

Programación

Programa tu sistema utilizando `skfuzzy` o `JFuzzyLite`. Incluye código fuente y capturas de pantalla con tus resultados.

Anexado*

Ponderación

A continuación, se muestra la ponderación de la evidencia correspondiente a Sistemas Difusos (incluye tanto la parte individual como la parte en equipo).

Concepto	Ponderación
R3.A1 (parte en equipo)	40
R3.A2 (parte individual)	40
Ejercicios en clase	20
Total	100

R3.A1 (parte en equipo)

Concepto	Ponderación
Descripciones	10
Variables lingüísticas	20
Conjuntos y funciones	20
Reglas	20
Programación	30
Total	100

R3.A2 (parte individual)

Concepto	Ponderación
Pregunta 1	30
Pregunta 2	20
Pregunta 3	10
Pregunta 4	10
Pregunta 5	30
Total	100

Interdata

```
In [176... import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl
```

Diseño

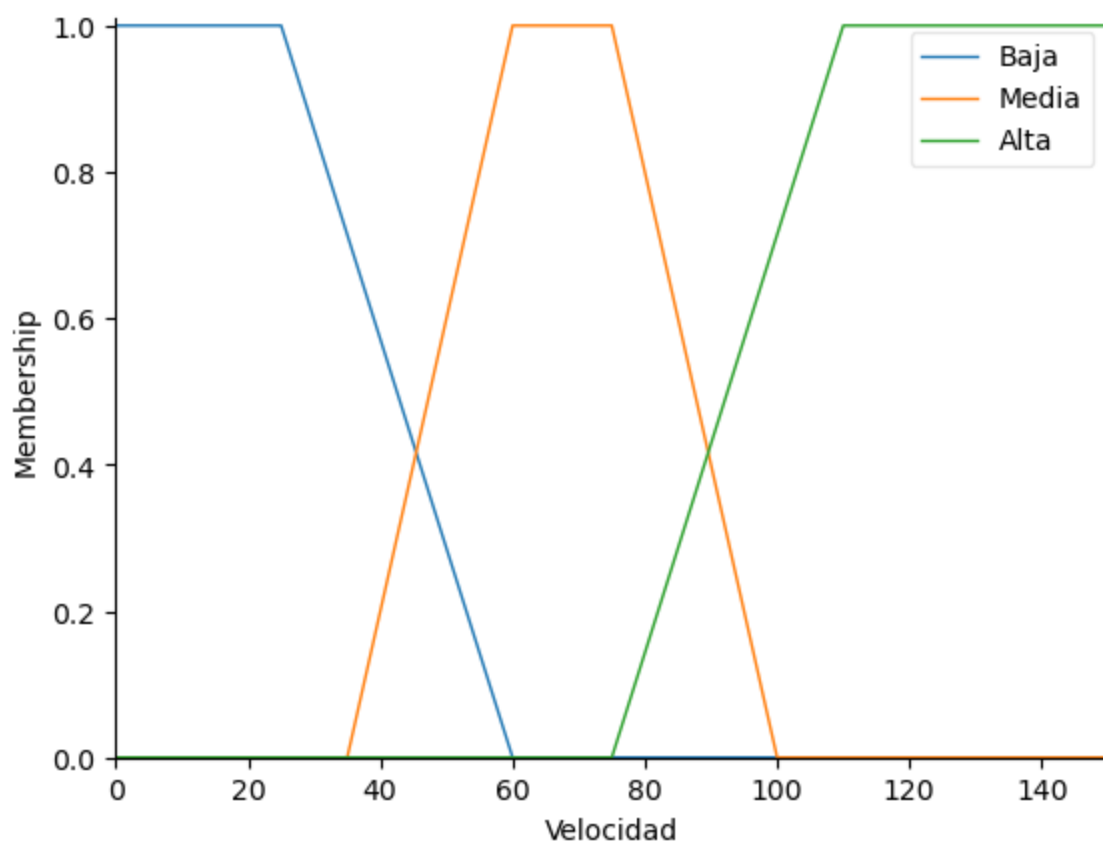
Variables

```
In [177... velocidad=ctrl.Antecedent(np.arange(0,151,1), 'Velocidad')
distancia=ctrl.Antecedent(np.arange(0,31,1), 'Distancia')
clasificacion=ctrl.Consequent(np.arange(0,13,1), 'Clasificacion')
```

Conjuntos difusos y funciones de membresía

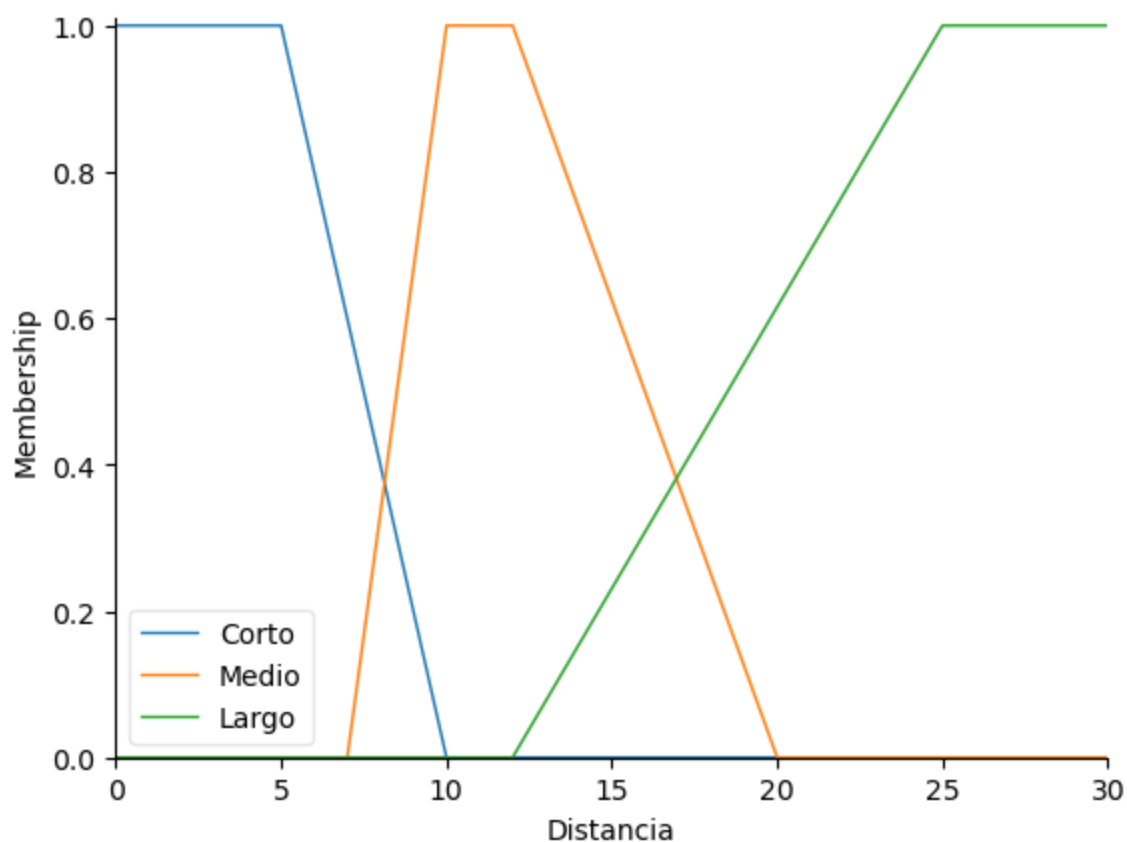
```
In [178... #velocidad
velocidad['Baja'] = fuzz.trapmf(velocidad.universe, [0, 0, 25, 60])
velocidad['Media'] = fuzz.trapmf(velocidad.universe, [35, 60, 75, 100])
velocidad['Alta'] = fuzz.trapmf(velocidad.universe, [75, 110, 150, 150])
```

```
In [179... velocidad.view()
```



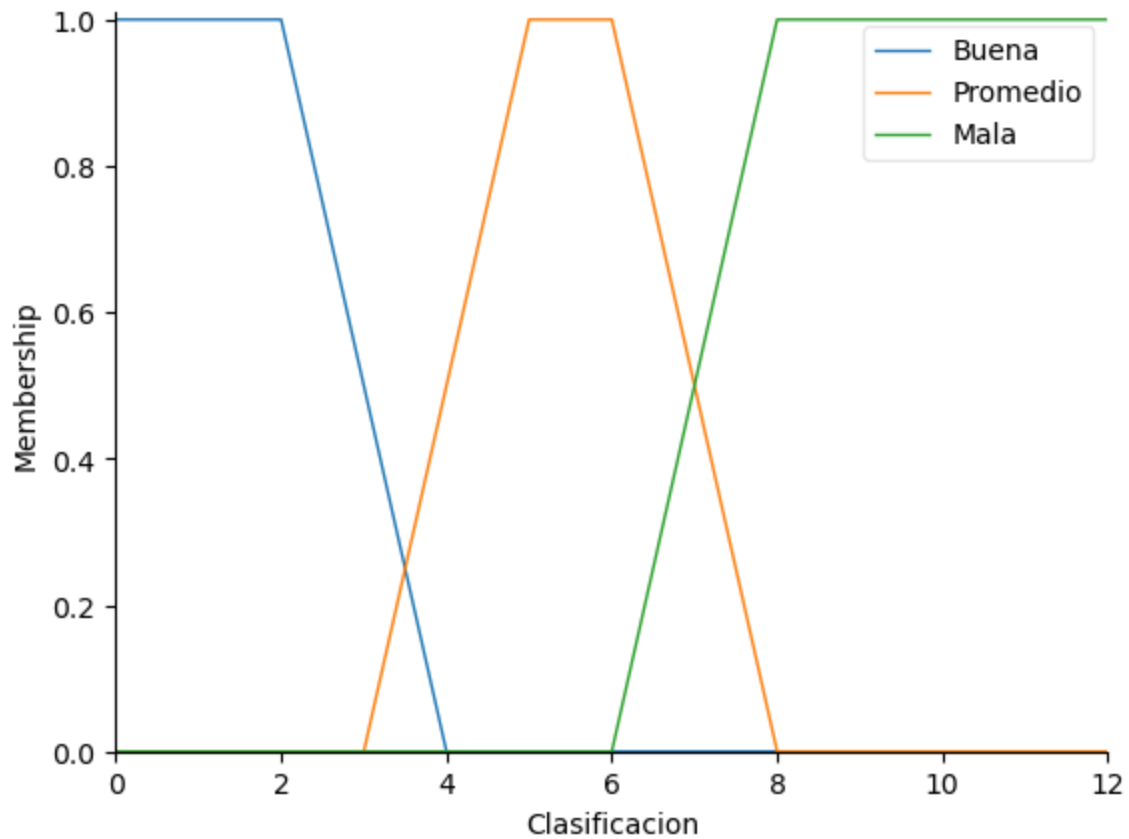
```
In [180... #Distancia
distancia['Corto'] = fuzz.trapmf(distancia.universe, [0, 0, 5, 10])
distancia['Medio'] = fuzz.trapmf(distancia.universe, [7, 10, 12, 20])
distancia['Largo'] = fuzz.trapmf(distancia.universe, [12, 25, 30, 30])
```

```
In [181... distancia.view()
```



```
In [182... #clasificaciones
clasificacion['Buena'] = fuzz.trapmf(clasificacion.universe, [0, 0, 2, 4])
clasificacion['Promedio'] = fuzz.trapmf(clasificacion.universe, [3, 5, 6, 8])
clasificacion['Mala'] = fuzz.trapmf(clasificacion.universe, [6, 8, 12, 12])
```

```
In [183... clasificacion.view()
```



Reglas

In [184...

```
rule1 = ctrl.Rule(velocidad['Baja'] & distancia['Corto'], clasificacion['Mala'])
rule2 = ctrl.Rule(velocidad['Baja'] & distancia['Medio'], clasificacion['Mala'])
rule3 = ctrl.Rule(velocidad['Baja'] & distancia['Largo'], clasificacion['Mala'])

rule4 = ctrl.Rule(velocidad['Media'] & distancia['Corto'], clasificacion['Buena'])
rule5 = ctrl.Rule(velocidad['Media'] & distancia['Medio'], clasificacion['Promedio'])
rule6 = ctrl.Rule(velocidad['Media'] & distancia['Largo'], clasificacion['Mala'])

rule7 = ctrl.Rule(velocidad['Alta'] & distancia['Corto'], clasificacion['Buena'])
rule8 = ctrl.Rule(velocidad['Alta'] & distancia['Medio'], clasificacion['Buena'])
rule9 = ctrl.Rule(velocidad['Alta'] & distancia['Largo'], clasificacion['Promedio'])
```

Proceso de inferencia

In [185...

```
rules=[]
rules.append(rule1)
rules.append(rule2)
rules.append(rule3)
rules.append(rule4)
rules.append(rule5)
rules.append(rule6)
rules.append(rule7)
rules.append(rule8)
rules.append(rule9)

sistema = ctrl.ControlSystem(rules)
```

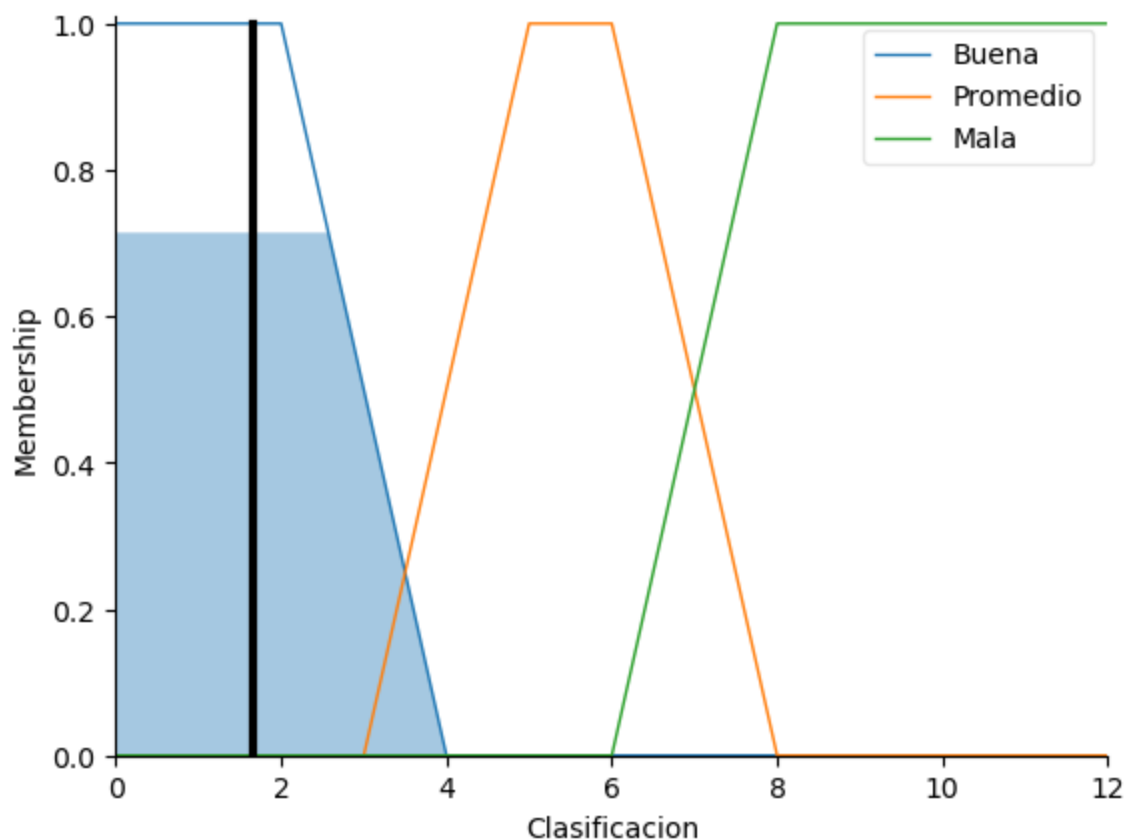
```
In [186...] caso=ctrl.ControlSystemSimulation(sistema)
```

```
In [187...] #Entradas nítidas
caso.input['Velocidad'] = 100
caso.input['Distancia'] = 5
```

```
In [188...] # Calcular salida
caso.compute()
caso.output['Clasificacion']
```

```
Out[188]: 1.668737060041408
```

```
In [189...] clasificacion.view(sim=caso)
```



```
In [190...] x = np.arange(0, 13, 1)
mfx = fuzz.trapmf(x, [3, 5, 6, 8])
defuzz_centroid = fuzz.defuzz(x, mfx, 'centroid') # Same as skfuzzy.centroid
defuzz_bisector = fuzz.defuzz(x, mfx, 'bisector')
defuzz_mom = fuzz.defuzz(x, mfx, 'mom')
defuzz_som = fuzz.defuzz(x, mfx, 'som')
defuzz_lom = fuzz.defuzz(x, mfx, 'lom')
xvals = [defuzz_mom,
          defuzz_som,
          defuzz_lom]
ymax = [fuzz.interp_membership(x, mfx, i) for i in xvals]
```

```
In [191...] import matplotlib.pyplot as plt

labels = ['centroid', 'bisector', 'mean of maximum', 'min of maximum',
```



```

        'max of maximum']
xvals = [defuzz_centroid,
         defuzz_bisector,
         defuzz_mom,
         defuzz_som,
         defuzz_lom]
colors = ['r', 'b', 'g', 'c', 'm']
ymax = [fuzz.interp_membership(x, mfx, i) for i in xvals]

# Display and compare defuzzification results against membership function
plt.figure(figsize=(8, 5))

plt.plot(x, mfx, 'k')
for xv, y, label, color in zip(xvals, ymax, labels, colors):
    plt.vlines(xv, 0, y, label=label, color=color)
plt.ylabel('Fuzzy membership')
plt.xlabel('Universe variable (arb)')
plt.ylim(-0.1, 1.1)
plt.legend(loc=2)

plt.show()

```

